

INVESTIGACION SPRING SECURITY

JORDAN ALEJANDRO HERRERA GOMEZ

JAIDER AUGUSTO NIÑO BADILLO

JULIO CESAR SILVA HERNANDEZ

ING. CARLOS ADOLFO BELTRAN CASTRO

UNIDADES TECNOLÓGICAS DE SANTANDER

PROGRAMACIÓN EN JAVA

BUCARAMANGA

2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN	3
2. TECNOLOGÍAS UTILIZADAS.	4
3. ARQUITECTURA DE LA SOLUCIÓN	5
3.1 Capa de modelo (Entidad)	5
3.2 Capa de Persistencia (Repositorio).....	5
3.3 Capa de Servicios	5
4. CONFIGURACION DEL MODULO DE SEGURIDAD (WEBSECURITYCONFIG)6	
4.1 Encriptación de Datos (BCryptPasswordEncoder).....	6
4.2 Gestion de Permisos (SecurityFilterChain)	6
4.3 Configuración del Login Formulario (formLogin):.....	6
5. CONSIDERACIONES.....	7

1. INTRODUCCIÓN

El presente proyecto se realizo en base al parcial realizado en el segundo corte correspondiente a “universidad”, agregándole un modulo de seguridad robusto. El objetivo principal es garantizar que solo usuarios autenticados puedan acceder al sistema protegiendo la información mediante estándares de la industria.

2. TECNOLOGÍAS UTILIZADAS.

- **Backend:** Java con Spring Boot.
- **Seguridad:** Spring Security para la autenticación y autorización.
- **Persistencia:** Spring Data JPA con Hibernate para la gestión de la base de datos.
- **Cifrado:** BCrypt para el hashing de contraseñas.
- **Frontend:** Thymeleaf como motor de plantillas y Bootstrap para el diseño responsivo.

3. ARQUITECTURA DE LA SOLUCIÓN

3.1 Capa de modelo (Entidad)

Se definió la clase Usuario, la cual representa una tabla de nuestra base de datos que se agregó al parcial para que esta entidad incluya atributos tales como username, password y rol se utilizaron las anotaciones de JPA tales como @Entity, @Table, @Column para el mapeo relacional.

3.2 Capa de Persistencia (Repositorio)

Se implementó la interfaz IUserarioRepositorio extendiendo de JpaRepository. Esto nos permite realizar operaciones CRUD básicas de forma automática. Además, se definió un método personalizado findByUsername(String username), esencial para que Spring Security pueda localizar al usuario durante el proceso de login.

3.3 Capa de Servicios

Esta capa la configuramos en dos

- **Gestión de Usuario:** Métodos para listar, guardar y eliminar. Aquí se integró el componente BCryptPasswordEncoder para asegurar que toda contraseña capturada desde el formulario sea cifrada antes de persistir en la base de datos, cumpliendo con las normativas de seguridad.
- **Seguridad (UserDetailsService):** Se implementó la interfaz de Spring Security para cargar los datos del usuario desde la base de datos y convertirlos en un objeto que el sistema de seguridad reconozca.

4. CONFIGURACION DEL MODULO DE SEGURIDAD (WEBSECURITYCONFIG)

La clase WebSecurityConfig es el centro de control de acceso de la aplicación. Utiliza Spring Security para definir quién puede entrar al sistema y qué recursos están protegidos.

4.1 Encriptación de Datos (BCryptPasswordEncoder)

Se define un @Bean para BCrypt, que es un algoritmo de hashing de contraseñas.

Esto lo que hace es que contraseñas como “12345” queden cifradas en la BD, cuando el usuario intenta logearse Spring security toma la contraseña ingresada y la cifra y la compara con la que está en la BD

4.2 Gestion de Permisos (SecurityFilterChain)

Este método es muy importante ya que gestiona la navegación

- **Acceso publico:** Se configuraron requestMatchers para liberar las rutas /login, /css/, /js/ e /images/. Esto es vital para que la página de inicio de sesión se vea correctamente (con estilos) antes de que el usuario esté autenticado.
- **Acceso restringido:** El comando .anyRequest().authenticated() indica que cualquier otra página de nuestro sistema universidad (cursos, asignaturas, usuarios) está bloqueada a menos que el usuario haya iniciado sesión exitosamente.

4.3 Configuración del Login Formulario (formLogin):

- **Página Personalizada:** Se especifica .loginPage("/login") para usar nuestro propio diseño de Thymeleaf en lugar del formulario por defecto de Spring.
- **Ruta de Procesamiento:** .loginProcessingUrl("/login") le indica a Spring que escuche las peticiones POST del formulario.
- **Redirección Exitosa:** .defaultSuccessUrl("/index", true) asegura que, una vez dentro, el usuario sea enviado siempre a la página principal de bienvenida.

5. CONSIDERACIONES

Realizamos una clase Datainitializer, la cual nos sirvió para mirar si estaba insertando las contraseñas a la BD encriptadas y para compararlas lo que hace esta clase por defecto es crear un usuario admin con contraseña 123456 las cuales nos servirán para realizar el login al sistema este mismo es el usuario que puede utilizar para acceder al sistema, luego en el apartado de Usuarios podrá crear uno nuevo o editarlo.